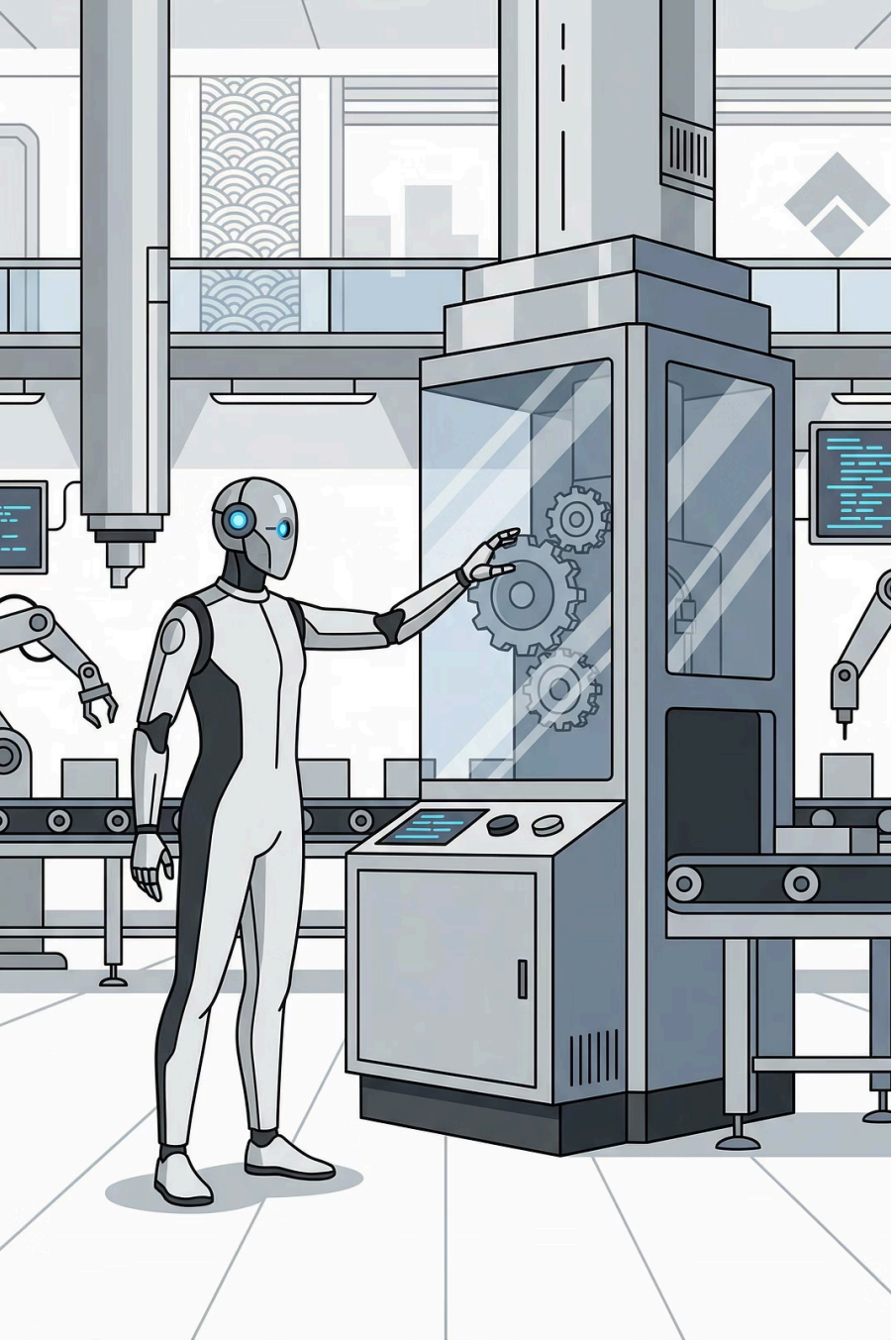


# SOP-Guided Robotic Task Planning

## A Deep Learning Approach to Incident-Driven Plan Generation in Manufacturing

Deep Learning Final Project • December 2025

# Problem Definition & Motivation

## The Challenge

**The Challenge**

In manufacturing environments today, humans must respond quickly and safely to machine incidents by following Standard Operating Procedures (SOPs). In the near future, humanoid robots will be deployed in factories and homes, where they will need to *understand and execute* these SOPs from natural language. However, current approaches struggle to reliably map incident descriptions and free-form SOP text into robot-executable actions, leading to increased downtime, higher operational costs, and limited autonomy

### Manual SOP Lookup

2–5 minutes per incident leads to increased downtime and significant operational costs

### Unstructured SOPs

Text instructions written for humans cannot be directly executed by robotic systems

### Keyword Search Fails

"Low pressure" vs. "Pressure drop" represent the same issue but different wording confuses systems

# Methodology

# Project Objective

Given a natural language incident description, our system automatically performs three critical operations to enable rapid robotic response:

### Semantic SOP Retrieval

Retrieve the most relevant Standard Operating Procedure using deep semantic matching rather than brittle keyword search

### Structured Plan Generation

Generate a structured, robot-executable plan in JSON format that can be directly consumed by robotic controllers

### Safe Skill Constraint

Constrain the plan to a fixed set of verified robot skills ensuring safe and predictable execution

# Target Environment

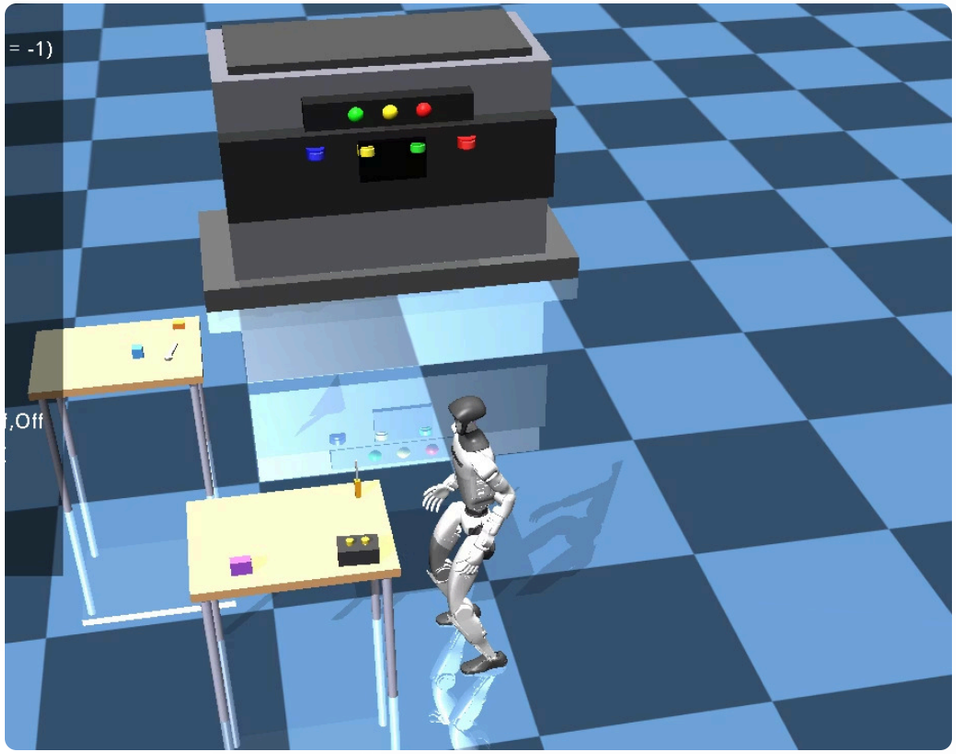
## Simulated Factory Cell

**Robot Platform:** Unitree G1 humanoid with locomotion and bimanual manipulation capabilities

**Scene Configuration:** Single manufacturing machine with surrounding work tables and components

## Physical Components

- 4 control buttons for machine operation
- 3 indicator lights for status monitoring
- 4 sensors for environmental readings
- 11 table objects (containers, tools, parts)




## Dataset Scope

**100 SOPs** covering machine control, manipulation workflows, and emergency procedures

**100 Incidents** each labeled with ground-truth SOP for evaluation

**7 Primitive Skills** forming the fixed action space for robot controller

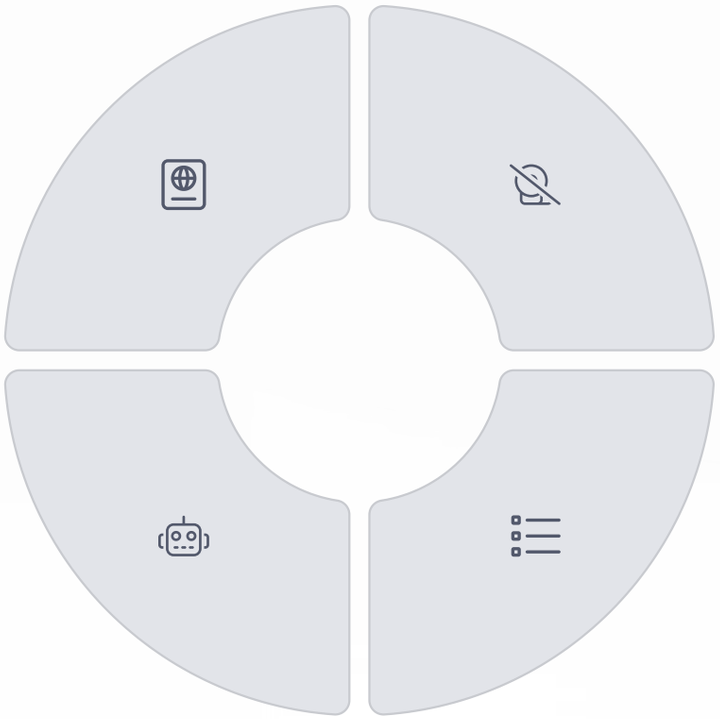
# Experimental Setup

### 100 SOPs

Manufacturing procedures spanning machine control (IDs 1–20), table manipulation (21–60), and complex workflows (61–100)

### 7 Skills

Fixed vocabulary ensuring all generated plans are executable and verifiable



### 100 Incidents

Each mapped to ground-truth SOP for evaluation, covering diverse failure modes and operational scenarios

### 774 Steps

Average of ~7.7 steps per SOP, all rewritten as compositions of 7 primitive skills

## Training Infrastructure

| Component | Hardware | Time | Epochs |
|---|---|---|---|
| Dual-Encoder BERT | Single GPU, 2070 Super | ~5 min | 50 |
| Flan-T5 + LoRA | Single GPU, 2070 Super | ~2 min | 10 |
| FAISS Index Build | CPU, i7 10thGen | <1 min | — |

# System Architecture Overview

01

### Incident Input

Natural language description of machine incident

02

### Dual-Encoder Retriever

BERT-based semantic matching with InfoNCE loss and FAISS indexing

03

### Top-K SOP Selection

Fast approximate nearest neighbor search over 100 SOPs

04

### Plan Generator

Flan-T5 with LoRA adapter produces structured JSON action plans

05

### Skill Executor

7 primitive skills mapped to robot controller primitives

End-to-end latency under 1 second enables real-time robotic incident response in production environments.

# Component 1: Semantic Retrieval

## Dual-Encoder Architecture

**Base Model:** bert-base-uncased (~110M parameters)

Two specialized encoders process incidents and SOPs independently:

- **Incident Encoder (Q):** Maps incident text to 768-dimensional embedding space
- **SOP Encoder (P):** Encodes SOP text into same embedding space

**Training:** Contrastive learning with InfoNCE loss function

**Indexing:** FAISS enables fast approximate nearest neighbor search at inference time

### InfoNCE Loss Function

Let $q$ = incident embedding (query), $k+$ = positive SOP embedding (ground-truth), and $\{ki\}$ = in-batch negative SOP embeddings with temperature $\tau = 0.05$

### Pre-computed Embeddings

SOP vectors computed offline enable O(1) lookup complexity

### Fast Retrieval

FAISS index supports sub-40ms retrieval across 100 SOPs

# Dual-Encoder Training Protocol

### Data Construction

Created ~100 positive (incident, SOP) pairs from labeled dataset. In-batch negatives provide 7–8 contrasting examples per training instance.

### Hyperparameters

50 epochs with batch size 8, learning rate 5e-5, 100 warmup steps, and temperature $\tau = 0.05$ for sharper similarity distribution.

### Convergence

Model converged around epoch 40 with stable Recall@1 metrics and consistent InfoNCE loss reduction.

## Dual-Encoder vs Cross-Encoder Trade-offs

| Criteria | Cross-Encoder | Dual-Encoder (Ours) |
|---|---|---|
| Accuracy | Slightly higher | High (Recall@1 = 0.96) |
| Latency | ~100 ms × N SOPs | < 40 ms total |
| Pre-indexing | Not feasible | Yes, with FAISS |
| Real-time use | Challenging | Suitable for robots |

# Component 2: Plan Generation

## LoRA Fine-Tuning Strategy

**Base Model:** google/flan-t5-base (~250M parameters)

**Output Format:** Structured JSON constrained to fixed skill vocabulary

### 0.5M Parameters

Only 0.2% of model parameters trained, enabling 2-minute training time

# Skill-Constrained Generation

## 7 Primitive Robot Skills

### walk_to

Navigate to specified location (machine, table, or position)

### press_button

Activate control button on machine panel

### wait

Pause execution for specified duration in seconds

### read_sensor

Query sensor value for pressure, temperature, or status

### pick

Grasp and lift specified object from surface

### place

Position object at designated location

### notify

Send message to human operator or technician

**Why Constraint Matters:** Limiting output to 7 verified skills ensures safety (only known actions), executability (maps to controller primitives), interpretability (operators understand plans), and validation (simple to verify before execution).

# Experimental and Results

# Retrieval Deep Dive

## Example Query Analysis

**Incident:** "Machine A pressure is low"

### Top-5 Retrieved SOPs

1. SOP-037: Gradual Pressure Build (0.98) ✓
2. SOP-051: Pressure Drop Emergency (0.97) ✓
3. SOP-061: Pressure Threshold Adjustment (0.96) ✓
4. SOP-001: Low Pressure Warning (0.95) ✓ **Ground Truth**
5. SOP-014: Pressure Calibration (0.94) ✓

**Key Observation:** All top-5 results are semantically relevant pressure-related procedures, even though ground-truth SOP ranks 4th. This demonstrates robust semantic understanding.

### Why Recall@5 Matters

In practice, multiple SOPs can be valid responses to a given incident. High Recall@5 (99%) means the system provides robust behavior under ambiguity, allowing operators or downstream planners to work with any top-K result.

# Plan Generation Results

**Incident Input:**

> "Yellow warning light is flashing, pressure gauge is low"

```
{
  "goal": "Low Pressure Warning",
  "steps": [
    {"skill": "walk_to",
     "args": {"target": "machine"}},
    {"skill": "read_sensor",
     "args": {"sensor": "pressure_sensor"}},
    {"skill": "press_button",
     "args": {"button": "blue_button"}},
    {"skill": "wait", "args": {"duration": 3}},
    {"skill": "notify",
     "args": {"message": "technician"}}
  ]
}
```

The Flan-T5 + LoRA model generates well-structured, executable plans that correctly sequence skills and specify appropriate arguments based on incident context.

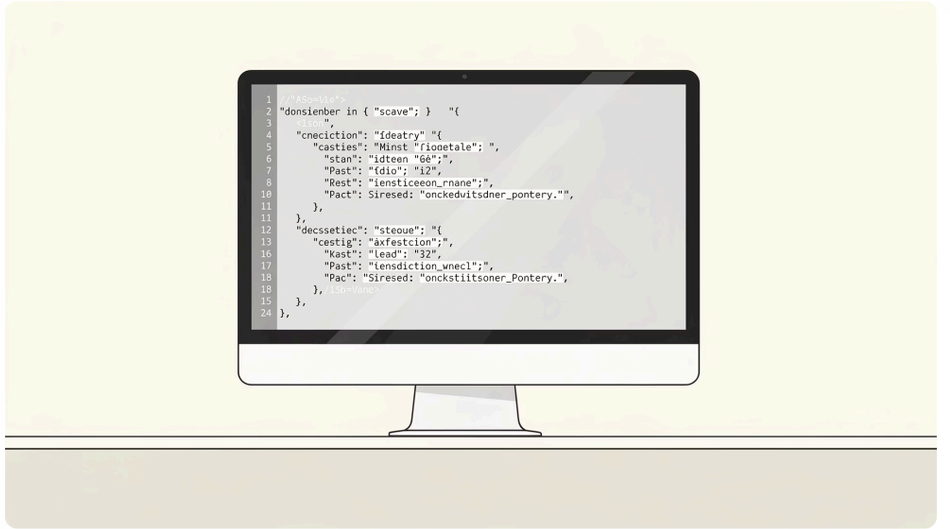
# JSON Plan Structure

## Input Prompt Format

Incident: Yellow warning light is flashing and pressure gauge is low.

SOP: Low Pressure Warning.
Condition: Yellow light flashing...
Steps: Walk to machine; Read pressure_sensor; Press blue_button...

Generate a skill-constrained JSON plan:



## Model Output Example

```
{
  "goal": "Low Pressure Warning",
  "steps": [
    {"skill": "walk_to",
     "args": {"target": "machine"}},
    {"skill": "read_sensor",
     "args": {"sensor": "pressure_sensor"}},
    {"skill": "press_button",
     "args": {"button": "blue_button"}},
    {"skill": "wait",
     "args": {"duration": 3}},
    {"skill": "notify",
     "args": {"message": "technician"}}
  ],
  "fallback": []
}
```

Plans average 5–7 steps and include structured arguments for each skill invocation, enabling direct execution by robot controllers.

# Evaluation Metrics

## Retrieval Metrics

### Recall@1

Percentage of queries where correct SOP appears in top-1 position. Critical for fully automated systems.

### Recall@5

Percentage where correct SOP appears in top-5. Accounts for cases with multiple valid SOPs.

### MRR

Mean Reciprocal Rank = $(1/N) \times \sum(1/rank_i)$. Measures overall ranking quality across all queries.

## Plan Generation Metrics

### Plan F1

$F1 = 2PR/(P+R)$ where P = precision, R = recall. Measures overlap between predicted and reference plans.

### Execution Success

Percentage of plans that execute without errors, with all skills valid and arguments well-formed.

### Valid JSON Rate

Percentage of outputs that parse as valid JSON with correct schema structure.

# Live Demo: End-to-End System

## Command Line Input

```
python src/cli/demo.py plan --q "bolt on machine is loose"
```

## System Processing

The system receives the natural language incident description, retrieves the relevant SOP, and generates an executable robot plan in JSON format.

## System Output

```
[22:35:06] Running pipeline: retrieve → plan
[22:35:18] Looking for trained models in: C:\Users\sikka\Documents\2025_IMP\Courses\Fall_25\DL\Project\artifacts\retriever_bert
           Incident encoder path: C:\Users\sikka\Documents\2025_IMP\Courses\Fall_25\DL\Project\artifacts\retriever_bert\inciden
           Incident encoder exists: True
           Loading trained incident encoder
[22:35:19] Retrieved top-5 SOPs by dual-encoder
             1. SOP-019: Bolt Tightening Procedure. Vibration_sensor high due to loos... (score: 1.1048)
             2. SOP-040: Wrench Tightening Inspection. Machine components need tighte... (score: 0.9422)
             3. SOP-020: Panel Screw Adjustment. Panel is loose on machine. Steps: Wa... (score: 0.9387)
           Selected SOP: SOP-019 - Bolt Tightening Procedure
           Condition: Vibration_sensor high due to loose bolts
[22:35:19] Planning for: Bolt Tightening Procedure
           ✓ Generated plan: 6 steps
             1. walk_to → {'target': 'table'}
             2. pick → {'object': 'wrench'}
             3. press_button → {'button': 'red_button'}
             4. place → {'object': 'tool', 'location': 'shelf'}
             5. read_sensor → {'sensor': 'vibration_sensor'}
             6. notify → {'level': 'tech'}
```


```
{
  "goal": "Bolt Tightening Procedure",
  "steps": [
    {
      "skill": "walk_to",
      "args": {
        "target": "table"
      }
    },
    {
      "skill": "pick",
      "args": {
        "object": "wrench"
      }
    },
    {
      "skill": "press_button",
      "args": {
        "button": "red_button"
      }
    },
    {
      "skill": "place",
      "args": {
        "object": "tool",
        "location": "shelf"
      }
    },
    {
      "skill": "read_sensor",
      "args": {
        "sensor": "vibration_sensor"
      }
    },
    {
      "skill": "notify",
      "args": {
        "level": "tech"
      }
    }
  ],
  "fallback": []
}
```

# Error Analysis & Limitations

## Top Retrieval Issues

### Semantic Overlap

"Pressure drop" vs "Pressure build" — similar wording but different operational intent

### Underspecified Query

"Machine issue" is too ambiguous and matches many SOPs equally

### Rare SOP

Very specific terminology with few training examples

### Negation Handling

"Machine not starting" — negation often misinterpreted by embeddings

## Top Plan Generation Issues

### Missing Steps

Skips critical steps like wait — mitigate by increasing max_length and improving training data

### Wrong Argument

press_button("green") instead of blue — needs more diverse training examples

### Invalid Skill

Generates "turn_on" outside skill vocabulary — add post-processing filter

### Truncation

Plan cut mid-step — raise output token limit to accommodate longer sequences

**Current Limitations:** Small dataset (100 SOPs) limits generalization; no reranker yet to refine top-K retrieval

# Conclusion & Key Contributions

### End-to-End Pipeline

Automated flow from incident text to executable robot plan with sub-second latency suitable for real-time manufacturing

### Efficient Training

Contrastive learning + LoRA enables strong performance on small datasets with minimal compute (~7 min total training time)

### ~336% Improvement (3x faster)

Domain-specific fine-tuning boosted Recall@1 from 22% to 96%, demonstrating critical value of adaptation

### Safe Execution

Skill-constrained output ensures plans are interpretable, verifiable, and executable by robot controllers

## Reproducibility Commands

```
python src/retrieval/build_dual_encoder.py  # Train retriever (~5 min)
python src/planner/train_planner_lora.py     # Train planner (~2 min)
python src/cli/demo.py plan --q "incident"   # Run full pipeline
```

**Future Directions:** Add cross-encoder reranker for top-K refinement • Scale to 1000+ SOPs and incidents • Implement hierarchical multi-stage workflows • Integrate with physical Unitree G1 for closed-loop validation

# References


- Karpukhin et al., 2020 – *Dense Passage Retrieval for Open-Domain Question Answering*, EMNLP.
- Hu et al., 2022 – *LoRA: Low-Rank Adaptation of Large Language Models*, ICLR.
- Chung et al., 2022 – *Scaling Instruction-Finetuned Language Models* (Flan-T5), arXiv.
- Johnson et al., 2019 – *Billion-scale similarity search with GPUs* (FAISS), IEEE BigData.
- van den Oord et al., 2018 – *Representation Learning with Contrastive Predictive Coding* (InfoNCE), arXiv.